

第2章 · 页面置换

1.1 虚拟内存

▸ 1.2 页面置换

概念

▸ 算法对比

Belady 异常

1.3 TLB

算法	策略	缺页率	实现复杂度	硬件支持
OPT (Belady)	淘汰未来最远使用的页	最低 (理论)	不可实现	—
LRU	淘汰最久未使用的页	接近 OPT	高 (需时间戳)	需硬件计数器
Clock (NRU)	循环扫描, 淘汰 reference bit=0 的页	接近 LRU	低	仅需 1 bit/页
FIFO	淘汰最早进入的页	中	最低	无
Random	随机淘汰	不稳定	极低	无

💡 要点

Linux 用的是什么? 两轮 Clock 算法 (类似 LRU 近似)。每个页有两个标志位: PG_referenced 和 PG_active。kswapd 后台线程周期性扫描并降级 inactive 页。

第2章 · 页面置换

1.1 虚拟内存

1.2 页面置换

概念

算法对比

Belady 异常

1.3 TLB

 BELADY 异常条件

$$N_{\text{faults}}(M + 1) > N_{\text{faults}}(M)$$

Belady 异常 (Belady's Anomaly)

对 FIFO 页面置换算法，增加物理页框数有时反而增加缺页次数。这是反直觉的——更多内存应该更好才对。

具体例子：访问序列

1,2,3,4,1,2,5,1,2,3,4,5

3 个页框 → 9 次缺页 | 4 个页框 → 10 次缺页
(反而更多!)

根因：FIFO 不关心访问频率，4 个页框时可能保留了很多"以后才用"的页面，导致空间利用率反而下降。

 要点

LRU 和 Clock 没有 Belady 异常。因为它们属于"栈算法" (stack algorithm) ——M 个页框时的驻留集始终是 M+1 个页框时的子集。

第3章 · TLB

1.1 虚拟内存

1.2 页面置换

▶ 1.3 TLB

▶ 概念

为什么重要

优化策略

📌 TLB 命中率影响

$EAT = h \times (T_{tlb} + T_{mem}) + (1 - h) \times (T_{tlb} + 2T_{mem})$

h=TLB 命中率 miss 时多一次内存访问读页表

TLB (Translation Lookaside Buffer)

MMU 内部的**硬件缓存**，存储最近使用的虚拟页号→物理页框号映射。本质是一个全相联的 SRAM 查找表，通常 64-1024 项。

TLB 下的有效访问时间

$EAT = h \times (T_{tlb} + T_{mem}) + (1 - h) \times (T_{tlb} + 2T_{mem})$

h = TLB 命中率 (通常 > 99%) | $T_{tlb} \approx 1$ cycle | $T_{mem} \approx 100$ cycles

💡 要点

h=99% 时， $EAT \approx 1 + 100 = 101$ cycles。**h=98% 时**， $EAT \approx 1 + 0.98 \times 100 + 0.02 \times 200 = 103$ cycles。TLB 命中率降 1%，性能损失约 2%。



全章总结

概念	一句话定义	关键公式
虚拟内存	进程地址空间与物理内存的解耦抽象	—
有效访问时间 (EAT)	考虑缺页后的平均内存访问时间	$EAT = (1 - p)T_{mem} + pT_{disk}$
多级页表	用树结构压缩页表，只为实际使用的区域分配	Size = 4MB/进程 → 按需分配
Page Fault	访问不在物理内存的页面时触发的中断	—
Belady 最优	淘汰未来最远使用的页面——理论下界，不可实现	—
LRU	淘汰最久未使用的页面——最优的可行近似	—
Clock 算法	循环扫描 reference bit, Linux 的实际选择	—
Belady 异常	FIFO 的悖论：更多页框 ≠ 更少缺页	—
TLB	页表缓存——99%+ 命中率是性能关键	$EAT \approx h \cdot 101 + (1 - h) \cdot 201$

公式墙

物理地址 = $PT[VPN] \times 4096 + offset$

$EAT = (1 - p) \times 100ns + p \times 10ms$

$EAT_{TLB} = h \times 101 + (1 - h) \times 201 \text{ (cycles)}$

$N_{faults}^{FIFO}(M + 1) > N_{faults}^{FIFO}(M) \text{ (Belady)}$

Q: 为什么 Linux 不用纯 LRU 而用 Clock?

Q: 多级页表有什么代价?

Q: Belady 异常只在 FIFO 中出现吗?

内存管理: 虚拟内存与页面置换

内存管理:虚拟内存与页面置换

第1章 · 虚拟内存

▸ 1.1 虚拟内存

▸ 概念

为什么需要

实现原理

1.2 页面置换

1.3 TLB

▴ 关键公式

$$EAT = (1 - p) \times T$$

第1章 · 虚拟内存

▸ 1.1 虚拟内存

概念

▸ 为什么需要

实现原理

1.2 页面置换

1.3 TLB

Slide 3

没有虚拟内存会怎样? 三个致命问题

第1章 · 虚拟内存

▸ 1.1 虚拟内存

概念

为什么需要

▸ 实现原理

1.2 页面置换

1.3 TLB

▮ 页表大小

$$\text{Size} = 2^{32} / 2^{12} \gg$$

Slide 4

1 CPU 发出虚拟地址

第2章 · 页面置换

1.1 虚拟内存

▸ 1.2 页面置换

▸ 概念

算法对比

Belady 异常

1.3 TLB

第2章 · 页面置换

1.1 虚拟内存

▸ 1.2 页面置换

概念

▸ 算法对比

Belady 异常

1.3 TLB

算法	策略
OPT (最佳置换)	淘汰未来最远使用的页面

第2章 · 页面置换

1.1 虚拟内存

▸ 1.2 页面置换

概念

算法对比

▸ Belady 异常

1.3 TLB

▴ BELADY 异常条件

$$N_{\text{faults}}(M+1) >$$

第3章 · TLB

1.1 虚拟内存

1.2 页面置换

▸ 1.3 TLB

▸ 概念

为什么重要

优化策略

▴ TLB 命中率影响

$$EAT = h \times (T_{\text{tlb}} + T_{\text{mem}}) +$$

h=TLB 命中率 miss 时多一次内存访问读页表

 全章总结

概念	一句话定义
虚拟内存	进程地址空间与物理内存的解耦抽象
有效访问时间 (EAT)	考虑缺页后的平均内存访问时间
多级页表	用树结构压缩页表，只为实际使用的区域分配
Page Fault	访问不在物理内存的页面时触发的中断
Belady 最优	淘汰未来最远使用的页面——理论下界，不可实现
LRU	淘汰最久未使用的页面——最优的可行近似
Clock 算法	循环扫描 reference bit，Linux 的实际选择
Belady 异常	FIFO 的悖论：更多页框 $\neq$ 更少缺页